

Praktikumsbericht zu Versuch 9

Von Daniel Baer (3235621)

a) Modellierung

Potenzielle Verbindungen bei n Boxen: $n * \frac{n-1}{2}$

Als Datenstruktur eignet sich eine Priority Queue.

b) Union-Find

Implementierung:

```
class DisjointSet:

    def __init__(self) -> None:
        self.parent: dict[int, int] = {}
        self.size: dict[int, int] = {}

    def make_set(self, value: int) -> None:
        self.parent[value] = value
        self.size[value] = 1

    def find_set(self, value: int) -> int:
        parent = self.parent[value]
        if parent != value:
            self.parent[value] = self.find_set(parent)
        return self.parent[value]

    def union(self, u: int, v: int) -> bool:
        root_u = self.find_set(u)
        root_v = self.find_set(v)

        if root_u == root_v:
            return False

        if self.size[root_u] < self.size[root_v]:
            root_u, root_v = root_v, root_u

        self.parent[root_v] = root_u
        self.size[root_u] += self.size[root_v]
        del self.size[root_v]
        return True

    def component_size(self, value: int) -> int:
        return self.size[self.find_set(value)]

    def component_sizes_desc(self) -> list[int]:
        return sorted(self.size.values(), reverse=True)

    def component_count(self) -> int:
        return len(self.size)
```

c) Kruskal

Schaltkreisgrößen (absteigend):

5, 4, 2, 2, 1, 1, 1, 1, 1, 1, 1

d) AoC – Teil 1

Schaltkreisgrößen (absteigend):

52, 43, 38, 30, 22, 22, 21, 21, 21, 20, 18, 18, 18, 17, 17,
16, 15, 14, 12, 12, 11, 11, 10, 10, 9, 9, 9, 8, 8, 7, 7, 7, 6,
6, 6, 6, 6, 5, 5, 5, 5, 4, 4, 4, ...

Produkt der drei größten: $52 * 43 * 38 = 84968$

e) AoC – Teil 2

Letzte Verbindung: (94118, 14262, 86867) --- (92049, 7915, 99919)

Produkt der X-Koordinaten: $94118 * 92049 = 8663467782$